

DRISHYAM

From Footage to Action

Tech Stack Decision Document

Recommended Technologies & Rationale

Document ID: DRISHYAM-TECH-001

Version: 1.0 | 13 September 2026

Confidential – For Development Use Only

1. Decision Summary

We recommend a pragmatic, production-proven stack optimized for real-time video + AI workloads, multi-tenant SaaS, and Indian deployment realities (edge preference, cost control, DPDP).

2. Frontend

Web: Next.js 14+ (App Router) + TypeScript + Tailwind CSS + shadcn/ui

Why: Excellent DX, SSR for dashboard shells, strong TypeScript support, large ecosystem, easy deployment (Vercel or self-host).

Mobile: React Native (Expo) or Flutter

Why: Single codebase for iOS/Android, mature push notification support, good performance for alert + clip review flows.

3. Backend

Primary API: NestJS (Node.js + TypeScript) or FastAPI (Python)

NestJS preferred if team is stronger in TypeScript and wants unified language with frontend + excellent WebSocket support. FastAPI preferred if AI team wants seamless Python model integration and rapid ML endpoint development. Both are acceptable; pick one and stay consistent.

4. AI / Edge

- Language: Python 3.10+
- Core: Ultralytics YOLO + custom fine-tunes, OpenCV, ByteTrack
- Runtime: ONNX Runtime / TensorRT / OpenVINO depending on hardware
- Hardware target: NVIDIA Jetson Orin / Xavier, industrial PCs with GPU, or AI cameras with NPU

5. Data

- PostgreSQL 15+ (primary)
- Redis 7+ (cache, pub/sub, rate limit, presence)
- S3-compatible object storage (AWS S3, MinIO, Cloudflare R2, or Indian cloud equivalent)

6. Auth & Security

- JWT access + refresh tokens (httpOnly cookies for web optional)
- bcrypt or argon2 for password hashing
- Role-based access control (RBAC) middleware

7. Real-time

- WebSocket (Socket.IO or native ws) for alert push
- FCM + APNs for mobile push

8. DevOps

- Docker for all services
- Docker Compose for local + early staging
- Kubernetes or managed container service for production scale
- GitHub Actions or equivalent CI/CD
- Structured logging (JSON) + metrics (Prometheus/Grafana or cloud equivalent)

9. Why Not Alternatives (Brief)

- Pure serverless for core detection path → cold starts and latency unpredictability hurt real-time guarantees.
- MongoDB as primary → relational integrity and complex queries around orgs/sites/alerts are better in PostgreSQL.
- Heavy microservices from day 1 → unnecessary complexity for MVP; start modular monolith + clear edge boundary.